

# On-board data handling systems

10

James W. Cutler<sup>a</sup> and Jacob Beningo<sup>b</sup>

<sup>a</sup>University of Michigan, Ann Arbor, MI, United States, <sup>b</sup>Beningo Embedded Group, LLC, Linden, MI, United States

## 1 Introduction

The primary purpose for nearly all space missions is to generate data. Data are the key driver and motivation for a space mission. In most instances, a space mission is designed around a primary payload that generates specific data which could include applications from Earth imaging, space environment research, astrophysics, heliophysics, or new technology demonstrations. Space mission data are by no means limited to the payload, even though they tend to be the largest dataset. Missions generate a wealth of additional data such as the telemetry, which catalogs the health and status of the spacecraft, its attitude, system and subsystem states, log files, and configuration settings. These data are managed by the on-board data handling system (OBDH).

There are at least four main functions on-board a spacecraft for handling data. These functions can be broken down into sources, sinks, processors, and transfers. *Sources* are subsystems that generate data. These could be individual sensors within a subsystem such as a magnetometer, or they can be an entire subsystem such as an attitude determination and control system (ADCS). When data are generated, those data often need to be stored. The destination for the data is known as a *sink* because it receives the data. Data find its way from a source to a sink through a data *transfer*. Again the transfer could be within a single subsystem from a sensor to local memory or it could be from a subsystem to another subsystem or from a subsystem into non-volatile memory storage. Finally, data are often *processed* on-board. The spacecraft must decide whether it will act immediately with the data or transfer it for processing in the future. A component or subsystem can carry out one or more of the four main functions of data handling.

Space systems are inherently distributed computing systems, and data handling is a critical issue to consider during the design stage. Failure to adequately consider data handling can result in lost mission data, overly intensive computing operations, power budget issues, and many other problems that will threaten mission viability. This chapter addresses these common data handling functions and CubeSat mission design. The four main functions of generation, storage, transfer, and processing are described. A fifth element, time, is discussed, which is crucial for understanding data. Various design considerations for OBDH systems are summarized, and an example reference design is provided. A summary of emerging trends concludes the chapter.

## 2 Component overview

Spacecraft OBDH systems are not one-dimensional data systems and, as such, cannot be considered at a single, architectural level. Instead, they should be visualized as multitiered systems, where data flow upward from lower level components to subsystems of increasing complexity until it arrives at the user’s fingertips. Data are generated at the lowest levels of the system by individual components, also known as sources. Data generated by a source are then transferred to a subsystem that aggregates the data and acts as a source for a higher-level subsystem that will eventually consume the data. For example, Fig. 1 shows a simplified ADCS that acts as a data sink for several magnetometers. The magnetometers act as data sources whose data are stored locally within the sensor memory. It is then transferred over a data bus to the ADCS which acts as a data sink. The ADCS subsystem stores the data it receives until they are needed. The ADCS acts as a data aggregator that collects data from a wide range of sources such as magnetometers, temperature, current, and voltage sensors. A subsystem may act on the data that are transferred to it by storing them, processing them locally, or transferring them to another subsystem that could do the same.

This same scheme also applies at a higher-level within the spacecraft. The spacecraft may contain several subsystems, all of which aggregate sensor data while acting as a data generator for other on-board, higher-level systems such as the flight computer. An example data flow diagram can be seen in Fig. 2. Notice that the flight computer is acting as a data sink to several subsystems such as the ADCS, payload,

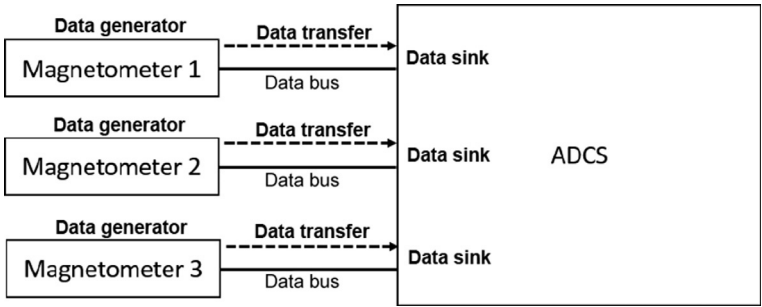


Fig. 1 Data flowing from sensors to a subsystem.

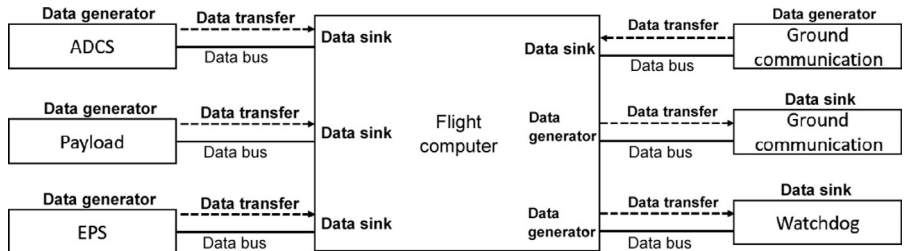


Fig. 2 Data flowing through the spacecraft.

and electronic power supply (EPS) while acting as a data generator for a watchdog subsystem. There is also a unique case with the ground communication system where the flight computer and the communication system act as data generators and sinks for each other.

Understanding how data flows through a component, a subsystem, and eventually through the spacecraft is critical for developing an appropriate hardware and software architecture for each of those components. As an introduction to the design process for on-board data handling, it is first discussed how data are generated, stored, and transferred.

## 2.1 Generation

Data on-board a spacecraft may be generated by several different sources depending on the overall system architecture. These include the flight computer, the attitude determination and control system, electronic power supplies, the fault management, and the telecommunication systems, but the primary data generator for a space mission is the payload. The payload on any given mission will typically generate data magnitudes larger than all other subsystems combined. Data generation can be characterized with four temporal qualities: deterministic versus event based and continuous versus variable rate. Each of these is briefly described in the following text.

*Deterministic* data collection occurs at regular or known time intervals with bounds on timing errors. A periodic telemetry beacon is an example. *Event-based* generation is opportunistic in nature and occurs when a stochastic event is detected. Data are generated in response to that event. Thresholding on telemetry points is an example event, where a particular value is reached for a data point, such as battery voltage. Additional sampling can occur to monitor the potential fault or take corrective action.

The data generation rates can approximately be divided into two categories: continuous or variable rate. Data generation over time can be at a *continuous rate* with sampling at fixed intervals. Low rate telemetry data are often generated in this manner as is payload data collected over the entire flight path of the mission. Variable rate sampling is useful for specialized data collection. Lower rates can be used to conserve resources such as storage space or power. Higher rates (sometimes called burst data collection) can provide higher fidelity measurements when needed.

Designers should consider at least four key attributes of data generation systems:

*Sample rate*—The number of times per time interval that data are generated or collected. Typically, this is given in terms of samples per second. Rates are determined based on expected features where sampling is done at least two times the expected signal frequency (the Nyquist rate [1]). Higher sampling rates provide greater resolution in time of signals.  
*Sample size*—The number of bits per sample. A larger number of bits per sample provides greater resolution in signal levels.

*Sampling time period*—The time over which data are sampled. There are two aspects to this. First, there is a low-level sample and hold time for when analog to digital converters make their measurements. The second, which is the main concern here, is the length of time that samples are taken at a particular sample rate. This is governed by the deterministic or event-base sampling described earlier.

**Table 1** Example telemetry points from subsystems.

| Subsystem              | Telemetry points                                                                                                                                                               |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Power system           | Battery voltages/currents/temperatures, on/off switch states, solar panel voltages/current/temperatures, battery heater status, output regulator temperatures/voltages/current |
| Attitude determination | Magnetic fields, rotation rates, sun sensor outputs (solar flux magnitude or sun vectors), star tracker outputs                                                                |
| Attitude control       | Controller mode, magnetic torque rod settings, reaction wheel speeds and directions, thruster power outputs                                                                    |
| Communications         | Bytes received, bytes transmitted, error rates                                                                                                                                 |
| Flight computer        | Time, command queue status, commanded on/off state, free memory, free storage, number of processes                                                                             |
| General purpose        | Voltage, current, temperature                                                                                                                                                  |

*Latency*—The time before data are made available for usage. Delays exist at all levels of system interaction. Designers should understand where the latencies are in their system which exists throughout the spacecraft between sensors and subsystems and for transferring data between generators and sinks. Low-latency requirements for systems like feedback control require real-time operating considerations.

In addition to payload data, data are also generated by spacecraft subsystems that are used to determine the spacecraft’s health, wellness, and status. Common measurements across these systems include power supply voltages, current draw, and temperatures. A nonexhaustive list of specialized telemetry is found in [Table 1](#).

One key common denominator among all generators is that they need somewhere to store their data. The different ways that data can be stored on the spacecraft will now be described.

**2.2 Storage**

Data storage is a fundamental element of computing [\[2\]](#) that provides a system with the ability to retain generated data and code. Requirements for storage vary greatly across a spacecraft’s subsystems and can be summarized with the following attributes:

- Size*—The number of bytes that can be stored.
- Volatility*—Storage persistence over a power cycle. Volatile storage loses the data when power cycled. Nonvolatile storage maintains the data through power cycles.
- Durability*—Data have a limited lifetime when stored. Volatility is a form of durability to power cycles. Electronic leakage current and radiation are additional degraders of storage. Durability describes the expected lifetime of the storage media.
- Write/read speed*—Storage has limits to the speed with which you can read and write to it. Typically, larger storage systems are slower to access.
- Latency*—There is a delay between when a read/write action is requested and when it is performed. When reading, this is a delay in the response of the data returning to the requester. When writing, it is a delay to when the data are physically written to the storage media.

Note: memory and storage are often used interchangeably by users. However, typically memory is used for short-term storage and storage is reserved for long-term storage. Short and long are not specifically defined, but memory is typically volatile RAM, and storage refers to nonvolatile, durable storage.

Location of the storage often constrains the design of storage systems. Storage that is close to sources and sinks of data is generally faster and has lower latency. Most subsystems have some small storage capability. If there is a processor, there will be volatile RAM and sometimes nonvolatile storage such as flash or MRAM. There are many different types of electronics chips that provide most forms of memory and storage to embedded systems; embedded multimedia cards (eMMC) are an example [3]. Larger storage systems may be an independent subsystem or part of a flight computer system. These storage systems provide retention of data for long-term archival on the satellite or as a buffer until they are downlinked to operations centers.

RAM is often used to store temporary data that will be used immediately or quickly transferred to another subsystem for use. Data stored in RAM are typically health and wellness information, subsystem state variables, and other subsystem specific information. When storing data in RAM, it is important to consider the radiation environment that the spacecraft will be operating in. Designers may need to protect the data's integrity by adding a cyclical redundancy check (CRC) to the data. If the data are considered critical, designers can also create duplicate copies of the data and store them in separate memory banks to improve the chances that the data remain intact.

Managing nonvolatile memory is complex. Many modern nonvolatile memory devices will automatically manage load wearing (equally distributing access to memory across the entire device) for a designer to improve the memory's durability, but a software driver is often required to interact with these devices. Designers need to carefully select their drivers. Open source software stacks such as FatFS may not be mission appropriate and are typically not fault tolerant. There are open source software stacks such as Yaffs, which have been used in NASA's Transiting Exoplanet Survey Satellite (TESS) if a commercial solution is not viable. An additional key design feature has been identified here; data need to be transferred from one system to another. This leads to the next section, data transfer between subsystems.

## 2.3 Transferring

In every spacecraft, data are transferred from one subsystem to another. That data may be health and wellness information, payload data, fault information or other data. Data are transferred through the spacecraft using a variety of mechanisms which can be categorized as hardware and software.

The hardware mechanisms used directly relate to the spacecraft bus architecture and the communication interfaces that are established between the various subsystems. The subsystems may be connected in a bus architecture that uses point-to-point communication such as a USART, Ethernet, or USB. Alternatively, a subsystem might use a multidrop or multipoint architecture such as RS-485 or CAN bus. The difference is that in a point-to-point topology the sender communicates directly with a single receiver while a multidrop topology allows for a sender to communicate with

multiple receivers, sometimes even simultaneously. These topologies are shown in Fig. 3.

There are two main characteristics that determine how effectively a communication bus can transfer data: bus type and data rate. There are two types of data buses, serial and parallel. Serial data present the data one bit at a time for transmission unlike parallel data that present an entire byte or more across multiple transmission lines at the same time. Serial is most often used today because it minimizes the number of transmission lines, which can reduce noise, transmission errors, cable size, and weight.

Serial communication protocols can also be either synchronous or asynchronous. Data that are sent synchronously are always with a clock to synchronize the communication between the devices. For example, the serial peripheral interface (SPI) uses two unidirectional transmission lines for master out/slave in (MOSI) and master in/slave out (MISO), which are paired with a clock (CLK) line to synchronize when to clock-in and clock-out a data bit. Asynchronous communication protocols do not use a clock to synchronize data transmission, which allows them to use fewer I/O pins. However, the lack of synchronization requires additional complexity such as adding start or stop bits to the transmission, which can decrease throughput. A USART is a classic example of asynchronous communication.

The interface that is selected to transfer a given set of data is greatly dependent on the amount of data to transfer and the data rate, which was discussed in the section on storage. The data rate is the rate at which the system can transfer data from one location to another and is highly dependent upon the communication interface that was chosen. Most modern interfaces today support at least 1 Mbps, as can be seen in Table 2, but the transfer rates can vary dramatically. For example, a standard USART found on nearly every microcontroller is capable of transfer speeds up to 4 Mbps. USARTs are often optimized for low-power operation, which can have advantages over more energy consuming interfaces such as USB or Ethernet. Designers need to carefully consider their communication interfaces data rate. While a USART can transfer at speeds up to 4 Mbps, the transmission line length can affect noise and error rates. It is not uncommon to see these very capable interfaces running at 115,200 bits per second (bps) or even 9600 bps! This is less a problem with noise than it is a problem with legacy thinking and supporting legacy communication stacks.

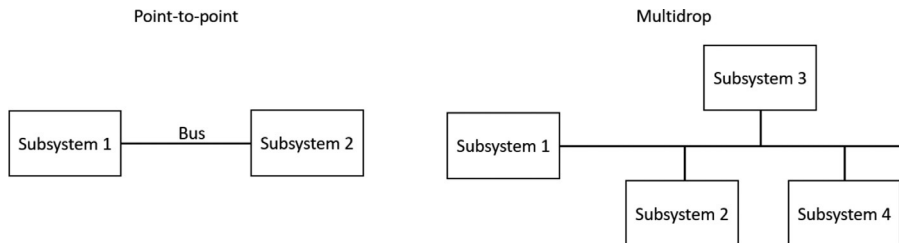


Fig. 3 Communication bus topologies.

**Table 2** Common communication interface topology and transfer rates.

| Bus interface | Data bus type | Topology       | Data rate (Mbps)   |
|---------------|---------------|----------------|--------------------|
| USART         | Serial        | Point-to-point | 4                  |
| RS-485        | Serial        | Multidrop      | 10                 |
| SPI           | Serial        | Multidrop      | 1–20               |
| QSPI          | Serial        | Multidrop      | 54–108             |
| I2C           | Serial        | Multidrop      | 0.1, 0.4, 1.0, 3.2 |
| CAN bus       | Serial        | Multidrop      | Up to 1            |
| USB           | Serial        | Point-to-point | 1.5–480            |
| Ethernet      | Serial        | Point-to-point | 10–1000            |

No matter how fast an interface may be, a major factor determining the actual transfer rates will be the software stacks that run the communication interface. A commonly used, layered architecture for communication stacks is the open systems interconnection (OSI) model. The model contains seven layers (Table 3) that standardize communication functions without regard to the underlying hardware. It is easy for designers to think only about the physical layer (layer 1), which is typically the processors low-level hardware peripheral such as SPI, I2C, USB, CAN, or Ethernet. There can be quite a few layers within the software stack that make transferring the data successful.

For example, Ethernet is a popular communication interface because of its high data transfer rates and its flexibility. A typical Ethernet transfer might involve the following software layers and protocols:

- Physical layer—10BASE-T Ethernet Peripheral
- Data Link—Ethernet
- Network—Internet Protocol (IP)
- Transport—Transmission Control Protocol (TCP)
- Session—Sockets Direct Protocol (SDP)
- Presentation—Secure Socket Layer (SSL)
- Application—File Transfer Protocol (FTP)

**Table 3** OSI model layer summary.

| Layer        |   |              | Function                                                    |
|--------------|---|--------------|-------------------------------------------------------------|
| Host layers  | 7 | Application  | High-level API's                                            |
|              | 6 | Presentation | Data translation between application and network service    |
| Media layers | 5 | Session      | Communication session management                            |
|              | 4 | Transport    | Data segment transmission between network points            |
|              | 3 | Network      | Multinode network management                                |
|              | 2 | Data link    | Data frame transmission between nodes on the physical layer |
|              | 1 | Physical     | Raw bit stream transmission and reception in hardware       |

The OSI model provides developers with a flexible layered architecture that can use a single model to communicate across several communication interface types and protocols (see Ref. [2] for a list of protocols by layer.) In many cases, developers can leverage existing software stacks to configure their software to use the desired transfer protocol(s).

## 2.4 Time

Space systems are inherently distributed, and time is typically used for coordination. Coordination includes such things as execution of commands at the proper time and time tagging of data to enable comparison with specifics. The scale of time distribution varies from subsystems on a satellite to interplanetary infrastructure.

A reference clock is chosen for the system, and distributed clocks synchronize to it. Usually there is a global reference standard as well that enables synchronization across multiple systems. Key attributes of a timing system include the following (Fig. 4):

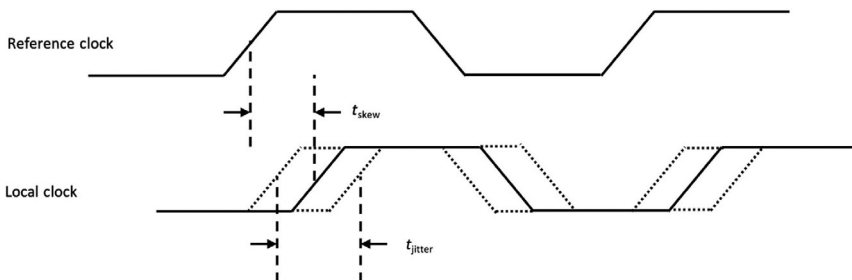
*Skew*—Error between local clock and the reference clock measured in units of time.

*Jitter*—A measure of error in the periodic signal that clocks output (often a sine or square wave).

*Drift*—The change in skew over time is called drift. External factors can cause deterministic drift such as radiation and temperature.

A variety of clocks are used on CubeSat systems. Simple clocks are crystal oscillators or 555 timers that provide a periodic signal. Counters or a processor can count these periodic signals and produce a time stamp that includes years, months, days, minutes, and seconds. GPS receivers provide a clock signal and time stamp referenced to the GPS satellite network. Advanced oscillators are temperature controlled to minimize drift. Recent advances in chip-scale atomic clocks enable highly accurate time stamps over large time horizons [4].

Some designs have external triggers that provide a reference event for timing. This event can be seen by portions of the system and aligned with local clock sources. The RAX mission described in Section 4 was able to use radar pulses from the ground with known time of flight for accurate time stamps. Other impulse events like lightning strikes can provide events as well. Some mission architectures use time



**Fig. 4** Example of clock skew and jitter of a local clock with respect to a reference clock.



synchronization during ground contacts—that is, synchronize mission elapsed time with universal time (UTC) or similar to compensate for drift and other effects.

There are multiple methods for *time tagging* or *time stamping*. Data sampling systems tag their output with references to clock signals. If the skew and jitter associated with the time tagging is low enough between samples, an initial absolute clock reference can tag the start of the data and offsets from this based on the clock used for following samples. If the time uncertainty is too great when sampling, the time can be written per sample to provide greater accuracy.

## 2.5 Processing

Once data have been generated, it is often processed in one form or another before it is stored in nonvolatile memory or transferred to another subsystem. How the data are processed will depend on what the data are used for, the real-time performance requirements, along with the available software and hardware. For example, data that are generated and immediately transferred to the flight computer for immediate control must be as close to real-time as possible and may be processed very little. On the other hand, payload data that are generated and not immediately used might be compressed to minimize memory storage use. Several operations that may be performed on the data can now be examined to understand where these operations are normally applied to for data in flight and at rest.

First, data that are transferred between subsystems will generally be encoded in a common packet format that allows commands and data to be propagated throughout the spacecraft. For example, a subsystem may generate health and wellness data and then encode them into a packet that is then transmitted to the flight computer for analysis. When the flight computer receives the packet, it needs to verify the packets integrity, decode it, and then it can act on the data that were received.

Data and commands can be encoded into many different types of formats, but many represent some type of framed, or packetized scheme. For example, one encoding scheme that uses point-to-point communication might provide a frame sync, an operational code (OP code), data length specifier, the data to be transferred and then a packet checksum to verify the packet's integrity, similar to the encoding scheme shown in Fig. 5. An alternative encoding scheme that could handle multidrop communication might instead leverage a starting frame sync, an address scheme, an OP code, data, checksum, and then a closing frame sync, similar to the encoding scheme shown in Fig. 6.

The encoding schemes used to command and transfer data are nearly limitless, but designers should consider the hardware interface, bus speed, data size to transfer, and real-time requirements to encode the data and generate checksums or cycle

|      |         |             |      |                 |
|------|---------|-------------|------|-----------------|
| Sync | OP code | Data length | Data | Checksum or CRC |
|------|---------|-------------|------|-----------------|

**Fig. 5** An example encoding scheme to command or transfer data for point-to-point communication.

|      |                   |         |      |                       |      |
|------|-------------------|---------|------|-----------------------|------|
| Sync | Address<br>scheme | OP code | Data | Checksum<br>Or<br>CRC | Sync |
|------|-------------------|---------|------|-----------------------|------|

**Fig. 6** An example encoding scheme to command or transfer data for a multidrop communication network.

redundancy checks (CRCs). These all need to be carefully balanced to meet system requirements. Note, many modern processors have fast, hardware support for CRC calculations, which can reduce needed resources.

Next, considerations can be made for encrypting the data. *Encryption* is typically only used on the communication link between the spacecraft and the ground station. Encryption provides a method for designers to protect their data by using an encryption key to convert their data or even their data packet into ciphertext. The only way that an entity would be able to decode the data is if they have a corresponding key. Encryption is not a guarantee that someone will not be able to decode data transmitted over RF. Designers need to ensure that they follow best practices for encryption such as a sufficiently long key and seed data that are changing in every packet, even if the packet data have not changed.

Encryption schemes can also be processing intensive. Implementing encryption should employ cryptographic hardware accelerators that are built into many modern microcontrollers and application processors. These hardware accelerators can often decrease the computing cycles required by 10–100 times. Hardware-based encryption can also simplify the software and decrease the power used during the operation.

Another process that can be performed on data is *compression*. Compression algorithms can be used to decrease the amount of storage space that a dataset will occupy. Compression is most often employed for payload data since it is typically the largest dataset generated on-board the spacecraft. When examining compression algorithms, there are two primary considerations that a designer should consider: first the compression/decompression speed, which is the number of megabytes per second (MB/s) that the algorithm can compress or decompress, and second the compression ratio, which is the ratio between the original file and the compressed file. The compression/decompression speed and the compression ratio are not necessarily in opposition, that is, just because the compression ratio is high it does not mean that the algorithm can process it quickly. Care needs to be taken to understand the trade-offs for various algorithms to balance their throughput, resultant data size, and the energy used for the process.

Finally, data that are retained may need to be cleaned or scrubbed. *Data cleaning* is the process of detecting and correcting or removing corrupt or inaccurate records from a dataset. For a spacecraft, this often pertains to the need to monitor memory locations and detect whether the data have become corrupted. Corruption could occur due to bit flips from radiation. It is particularly useful to include a CRC with data that are in memory in order to detect if the data have become corrupt in some way.

Once the data corruption have been detected, the data can be scrubbed. *Data scrubbing* is the act of detecting where the corruption is and correcting it. One technique that is often employed in spacecraft is to maintain multiple copies of critical data that is

resident in volatile memory. The data are stored in different memory locations that are not within the same memory segment. Periodically a background task will review the memory state to detect if the memory has become corrupted. If the data have corrupted, the back-up memory is tested and if its integrity is intact, then an update is performed to repair the corrupted memory.

### 3 Design considerations

An important part of the design process is developing the *architecture*, which is the distribution of data handling functions in the spacecraft. Generation is distributed by nature as data are generated throughout all subsystems. Processing and storage can be centralized or distributed depending on a variety of constraints related to the amount of data and transfer rates, as described in the preceding text. When designing any system, mapping out how data flow through and are processed in the system is absolutely critical to developing a system and software architecture that not only meets mission requirements but also provides flexibility and scalability to the design. In this section, additional tools are provided to help develop the architecture and other important design considerations.

There are four budgets to develop and analyze carefully when architecting the OBDH systems: data, processing, storage, and transfer. Each of these budgets map to the design elements described earlier. These budgets are not mutually exclusive but instead must be carefully balanced against each other to design an efficient and fully functional data handling system.

A *data budget* identifies and describes the data that are generated by each subsystem. The data budget helps designers understand what data are generated by their subsystems, how often it is generated, how much of it there will be, and determine the frequency and usefulness of that data. An example summary table is shown in [Table 4](#). The data budget provides an estimate of the amount of raw data that will be generated. These values can be tuned by adjusting sample rates and resolution at the expense of accuracy in values.

**Table 4** An example data budget for an active magnetic attitude determination and control system.

| Data           | Data type (bytes) | Data size (bytes) | Sample rate (Hz) | kB/s    | Data per orbit (kB) |
|----------------|-------------------|-------------------|------------------|---------|---------------------|
| Magnetometer   | 2                 | 6                 | 2                | 0.012   | 63.3                |
| Temperature    | 2                 | 6                 | 1                | 0.006   | 31.7                |
| Torque current | 2                 | 6                 | 5                | 0.029   | 158.2               |
| Torque voltage | 2                 | 6                 | 5                | 0.029   | 158.2               |
| Status packet  |                   | 64                | 0.1              | 0.00625 | 33.75               |

Raw, unprocessed data do not exist in this form for very long within most systems. The data are processed by the subsystems almost immediately whether it is to encode the data in a structure or compress it for storage. The data may even be generated, checked for faults, and then simply discarded. Designers can examine the data that they are generating and then list out how that data will be processed using a *processing budget*, as shown in [Table 5](#).

Notice that for the processing budget, the processing time is estimated in relation to the CPU utilization. *CPU utilization* is the percentage of time that the process is expected to take from the CPU between 0% and 100%. The CPU utilization for each data item is normalized to account for the various sample rates, data sizes and the algorithms that will be executed on the data. Designers should also keep in mind that in order for a CPU to be responsive and not overwhelmed, the target CPU utilization for all system tasks, processes, and algorithms should be approximately 70% or less. CPU utilization estimates are often highly prone to error due to coding complexity and variation and can change based on the processor architecture, execution mode, and clock speed. Estimations should be backed up with measurements during development.

Once the designer has established how much data will be generated and how it will be processed and handled, the storage budget will need to be examined. As seen earlier, there are two main storage mediums, volatile, and nonvolatile memory. If data will be stored locally in volatile memory, that data may require additional processing such as cleaning and scrubbing to ensure that its integrity remains intact (note, this was

**Table 5** An example processing budget from the active magnetic control system.

| Data           | Sample rate (Hz) | kB/s    | Data per orbit (kB) | Process                                                                                                                                           | Estimated CPU utilization (%) |
|----------------|------------------|---------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Magnetometer   | 2                | 0.012   | 63.3                | <ul style="list-style-type: none"><li>• Running average (5 samples)</li><li>• Encoded for health and status</li><li>• Logged in storage</li></ul> | 2.0                           |
| Temperature    | 1                | 0.006   | 31.7                | <ul style="list-style-type: none"><li>• Encoded for health and status</li></ul>                                                                   | 0.1                           |
| Torque current | 5                | 0.029   | 158.2               | <ul style="list-style-type: none"><li>• Encoded for health and status</li></ul>                                                                   | 0.1                           |
| Torque voltage | 5                | 0.029   | 158.2               | <ul style="list-style-type: none"><li>• Encoded for health and status</li></ul>                                                                   | 0.1                           |
| Status packet  | 0.1              | 0.00625 | 33.75               | <ul style="list-style-type: none"><li>• Transferred to flight computer, stored in nonvolatile memory log</li></ul>                                | 5.0                           |

not included in Table 5). If data will be stored in nonvolatile memory, the data may need to be compressed before storing.

The *storage budget* enables development of storage requirements for a system. It captures what data will be stored and the length of time for storage, which allows the required storage capacity for the subsystem or spacecraft to be determined for the entire mission. The storage budget also specifies the lifetime for the data stored, which is the amount of time the data will reside in storage before it is deleted. Missions that downlink data often, or at a high rate will be able to free storage capacity, which will decrease the total storage required for the mission.

The storage degradation rate should also be considered. Nonvolatile memory, such as flash memory systems, typically degrades out after a specified number of read/write/erase cycles. Some flash media will automatically perform wear leveling to try and maximize the storage media useful life and minimize degradation. The degradation or failure rate for a generic storage device can be calculated using the following procedure:

1. Based on the processing and generation rates, calculate the amount of read writes associated with the data.
2. For a given storage medium, determine the limits on reading/writing. (Note also that storage has a read/write speed which limits access rates as well.)
3. Develop a concept of operation that allots the budget to various storage elements.
4. Review the storage medium specifications for maximum read, write, and erase cycles.
5. Ensure that not only the expected read/write/erase cycles are less than the specifications but also there is at least a 20% buffer.

Storage degradation is not the only issue to consider. The storage media may fail as well and backup strategies are necessary to continue the mission during the presence of failures.

The *transfer budget* examines several different transfer mechanisms to ensure there is enough bandwidth to move data around subsystems and the spacecraft effectively. First, there is a transfer budget within a subsystem to transfer data from sensors into volatile memory and then into any nonvolatile storage. Next, there is a transfer budget for transferring data between subsystems. Finally, there is a transfer budget for the data to go from the spacecraft to the ground. The mission concept of operation governs how and when these transfers occur. Thus a full systems perspective is needed to budget data transfer appropriately to meet mission requirements.

These budgets are not mutually exclusive. They are interdependent and must be optimized together in order to maximize the amount of data that can be downloaded to the ground. Increasing the amount of data generated will affect the processing budget. It may also affect the storage and transfer budget as well.

A useful set of properties to consider when designing a data handling system are atomicity, consistency, isolation, and durability. Collectively, these are known as ACID [5], and they were derived during the development of databases. *Atomicity* means that the transaction completely succeeds or completely fails. *Consistency* ensures that the data will be valid. *Isolation* ensures that all transactions are isolated and will not affect others. *Durability* means that even if a failure occurs, the

transaction will still remain. Implementations of these transactions in a data handling system will promote robust operation.

At first, this might all seem overwhelming, but designers do not need to start from scratch; there are a variety of standards to review and consider for data handling. For example, the Consultative Committee for Space Data Systems (CCSDS) publishes a wide range of standards including spacecraft on-board interface services. Standards such as IP-based protocols from the Internet can also be used. These can provide design architectures with example code and existing solutions that ensure a shorter development time and a more robust solution.

There are other system-level considerations that are not covered in detail, for example, power usage. There are trade-offs between the processing technology used on a subsystem with the available energy for a mission. The more data that are available to be transferred or processed, the higher the power usage will be for the system. The power budget should be used to guide the design of the data handling system as well.

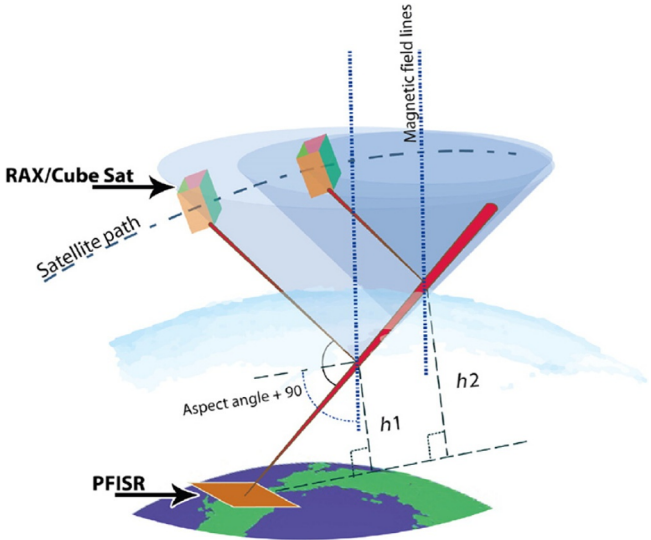
In summary, design can be guided by seeing this as an optimization process; the goal is to optimize data downloaded. To achieve this optimization, designers trade-off between the amount of data they generate, store, process, and transfer based on the overall mission objectives and available power budget.

## 4 Design example—RAX

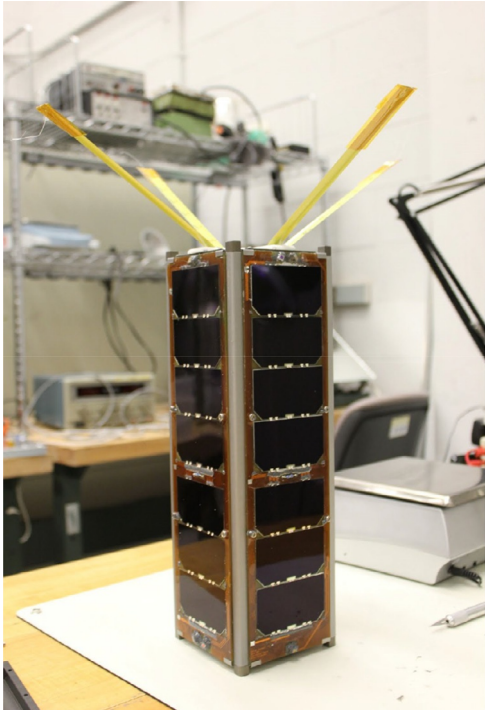
As an example, consider the Radio Aurora Explorer (RAX) mission, a partnership by SRI International and the University of Michigan [6]. The primary mission of RAX was to study the polar ionosphere and the plasma irregularities that develop along the magnetic field lines. Measurements were made with a bistatic radar system; see Fig. 7. A ground radar illuminated the ionosphere and RAX received the signal scatter from the irregularities. Two satellites were launched: RAX-1 on November 19, 2010 on-board a Minotaur-IV rocket from Kodiak, Alaska, and RAX-2 on October 28, 2011 on-board a Delta-II rocket from Vandenberg US Air Force Base, California. RAX-2 detected and measured irregularities during multiple experiments over Alaska and Canada (Fig. 8).

RAX captured the “spirit” of many early CubeSat missions; a novel science mission with limited funding and an aggressive schedule centered in a university research environment. It was the first nanosatellite missioned funded by the US National Science Foundation to study space weather [7]. Beyond the science the mission itself was an experiment to see if the constrained CubeSat platform could be used for novel, science measurements. The schedule was aggressive, with only 12 months from start to delivery for launch of RAX-1. Funding was constrained to US\$ 900K for payload and satellite development. These constraints encouraged creative, higher-risk solutions to perform the radar science mission.

The data handling requirements for RAX were driven by the payload radar receiver. It generated 32Mbps of data; its output was 16 bits wide, clocked at 1 MHz with data on each clock edge. Each radar experiment lasted approximately 5 minutes with a science goal of one experiment per day. Thus 1.2GB were generated



**Fig. 7** The radar experiment performed by the RAX mission.



**Fig. 8** The RAX-2 CubeSat.

per day. The set of driving requirements and constraints for RAX are summarized in the following text:

- (1) Collect data at 32Mbps, store it, and process 1.2GB per day.
- (2) Time tag data to better than 20 $\mu$ s.
- (3) Limit system size to a 3U CubeSat.
- (4) Meet the aggressive schedule and funding constraints.

At RAX design time, there were no off-the-shelf CubeSat processor systems that could handle the payload data requirements. Thus the team developed a distributed architecture to leverage existing capabilities when possible and use custom developed systems when needed, which is shown in Fig. 9. The system consisted of a specialized payload interface that collected and stored high speed payload data and provided an interface to embedded processor-based flight computer.

The RAX OBDH architecture is shown in Fig. 10. Processing and data storage were distributed through the satellite to provide a flexibility and redundancy. Each of the subsystems is described in detail in the succeeding text.

A custom *payload interface module* (PIM) was developed to collect and store data from the receiver and provide access to the data to other computing elements of the

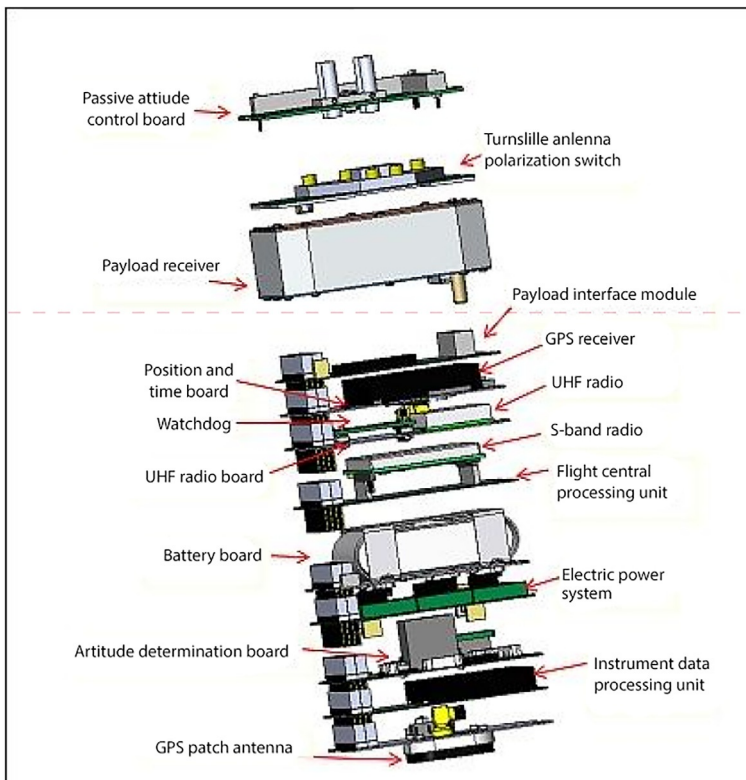
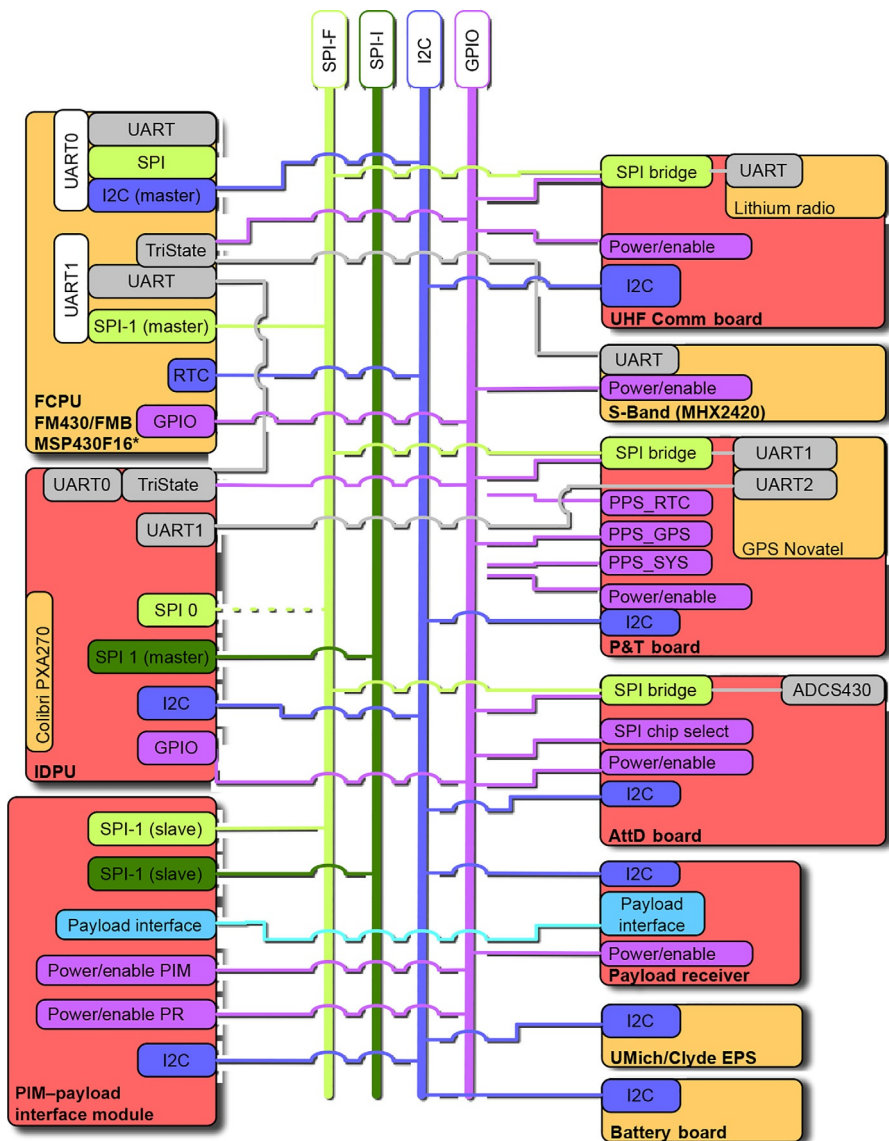


Fig. 9 The RAX board stack.





**Fig. 10** The RAX OBDH architecture.

system. The PIM contains a gate array clocked at approximately 50 MHz. The frequency is slightly lower than 50 MHz to prevent electromagnetic interference from 50 MHz harmonics appearing in the science data. Data were stored to 4 GB of mirrored flash memory bank and packetized with checksums and time tags inserted based on timing and pulse-per-second output from the position and time system. The interface module is accessible by both the flight computer and the instrument data processing

unit through dual SPI busses. It provides a shared storage system to enable large file transfer between the flight computer and the processing unit. The processing unit retrieves payload data from the interface module and copies processed data back to it. The flight computer retrieves these files and stores them on flight-computer-based storage for later downlink. In a similar manner, files are transferable from the flight computer to the processing unit to enable code updates on the processing unit.

The *instrument data processing unit* (IDPU) processes radar data to reduce downlink requirements. On-board processing sorts through time series receiver data stored on the payload interface module and extracts data with radar pulses of interest. The segmented data are then processed with radar analysis code to calculate signal-to-noise ratios and range delays. The processing unit then stores the data on the payload interface module for downlink by the flight computer to the science operations center. It also has direct interface to the GPS unit for enhanced configuration and monitoring of GPS performance.

The *flight computer* (FCPU) was a low-power embedded processor system based on the Texas Instruments MSP430 microprocessor and was developed internally by the RAX team. It performs typical flight computer functions such as telemetry collection, ground command processing, experiment sequencing, scheduled command execution, data storage, and communication through the radio systems. An integrated secure digital (SD) card provides 2 GB of data storage. Custom SD-card drivers were written because off-the-shelf libraries were too large in terms of code space; the libraries would not fit on the MSP430 with the other flight-specific code. An external watchdog timer system provided fault detection and recovery of flight computer transient failures [8]. The watchdog monitors a heartbeat signal from the computer and reboots it if a failure is detected. It can also be commanded from the ground through the UHF radio to force power resets of the flight computer. The watchdog itself is periodically power cycled by a 555-timer circuit to clear any internal transient errors. Flight software was custom written for RAX in C and ran on the Salvo operating system [9]. All available code space was used.

The *position and time system* (PTS) provided spatial and temporal information for the satellite to improve resolution of the irregularities and their fine scale features. The primary position and time source was an on-board GPS receiver. Detailed simulations of proper antenna placement in the context of the attitude control scheme were performed to estimate expected on-orbit performance [10]. A secondary timing system, based on a real time clock, provides time synchronization when GPS is not available, such as during planned GPS power off for energy conservation. The clock provides millisecond accuracy and maintains time during satellite reboots and resets. A pulse-per-second signal, sourced by either the GPS or the clock, is distributed throughout the satellite to synchronize data collections. The payload interface module and attitude determination system then use the signal to time tag data collections. A secondary ground-based position determination system uses publicly available two-line element sets published by the US government. Two-line element-based position error is expected to be less than 1 km within several days after the element set epoch [11].

The FCPU coordinated serial connections to all subsystems. It has minimal processing capability and data storage. Telemetry measurements are made throughout

the satellite with I2C enabled ADCs. It is collected by the FCPU and stored on SD-cards. SPI, I2C, and UARTs are used for communication. Point-to-point links were made with UARTs. The I2C bus was used for distributed telemetry collection. The SPI bus was used for high speed, point to point links. The FCPU configures the system for payload data collecting. It enables the payload to take data and commands the PIM to collect data. When the data collection is over, the FCPU commands the IDPU to copy data from the PIM, which is stored on the IDPU flash storage. The IDPU then processes the data. The FCPU transfers the processed data for storage until downlink to ground occurs.

The FCPU system on the RAX CubeSat's completed its mission to record, process, and download the payload data. The specialized PIM and IDPU provided the proper transfer rate and computational abilities. Redundancy in the storage capabilities allowed for multiple data storage configurations that prolonged mission flight in the presence of component failures.

## 5 Emerging trends

The emergence of CubeSats as a viable tool for scientific measurements is leading to increased innovation for on board data handling systems. Despite the inherent constraints imposed by the CubeSat form factor, many CubeSat missions have advanced OBDH systems due to their risk posterior; given cost and schedule pressure, the missions accept higher risk, more capable processing systems. There are several emerging trends that are increasing CubeSat OBDH capabilities:

*Parallel processing*—As processing systems decrease in physical size, missions are leveraging multiple processors for a variety of parallel processing applications. Computer on modules (COMS) provide off-the-shelf computers that can be parallelized into complex computing platforms [12]. Multicore processors provide on-chip parallel processing opportunities. For example, a low-power, real-time core can be used for continuous monitoring and low-level details. When an event of interest occurs, the second core can be woken up to increase processing capabilities and later return to a low power mode.

*Advanced coprocessors*—As processor technology advances, customized processing is available for specific computation needs. Graphical processing units (GPUs) are used terrestrially to provide high speed imaging processing capabilities. These capabilities are being deployed in mobile technology to provide advanced facial recognition and image enhancements. Field programmable gate arrays (FPGAs) provide software configured hardware, which runs faster than general purpose architectures but not necessarily as fast as custom processing [13]. Many FPGAs are capable of in-flight reprogramming to enable algorithmic upgrades.

*Edge computing*—The location of processing is a design variable. Recently, *orbital edge computing* has addressed the location of computational capabilities within low-Earth orbiting networks [14]. As intersatellite links improve, CubeSats will be able to share and distribute processing to other nodes in the network.

*Hypervisors*—Operating systems (OS) can be decoupled from hardware through virtualization. Hypervisors virtualize the hardware and provide a common, abstract hardware interface to the OS. This allows *virtual machines* to be run on distributed, heterogeneous hardware [15]. They can be used for fault tolerance and recovery as well [16].

*Advanced algorithms*—Machine learning (ML) techniques are being used to improve data management and processing. ML has been used to automatically detect events of interest and enable advanced data collection [17]. Future algorithms can be used to identify additional interesting or unexpected features in data. These capabilities should reduce required data download and latency of event detection.

These advancements are leading to novel OBDH systems for CubeSats that are providing enhanced capabilities and greater fault tolerance.

## References

- [1] A. Grami, *Introduction to Digital Communications*, Academic Press, 2016, .pp. 217–264. <https://www.elsevier.com/books/introduction-to-digital-communications/grami/978-0-12-407682-2>.
- [2] J.L. Hennessy, D.A. Patterson, *Computer Architecture: A Quantitative Approach*, Elsevier Science, Netherlands, 2017.
- [3] JEDEC, eMMC 5.0 Specification (JESD84-B50), Available from: [www.jedec.org](http://www.jedec.org), September 2013.
- [4] S. Nydam, J. Anderson, N.S. Barnwell, J. Peace, F. Pistella, T. Ritz, S. Roberts, et al., A compact optical time transfer instrument for ground-to-space synchronization of clocks, in: *AIAA SPACE and Astronautics Forum and Exposition*, 2017, p. 5381.
- [5] J. Gray, Why do computers stop and what can be done about it? in: *Symposium on Reliability in Distributed Software and Database Systems*, 1986, pp. 3–12.
- [6] H. Bahcivan, J.W. Cutler, Radio aurora explorer: mission science and radar system. *Radio Sci.* 47 (2) (2012) RS2012, <https://doi.org/10.1029/2011RS004817>.
- [7] T. Moretto, CubeSat mission to investigate ionospheric irregularities. *Space Weather* 6 (11) (2008) <https://doi.org/10.1029/2008SW000441>.
- [8] J. Beningo, *A review of watchdog architectures and their application to Cubesats*, Beningo Embedded Group, Linden, MI, 2010.
- [9] A.E. Kalman, Overview of the CubeSat Kit™, in: *Proceedings of the 18th Annual AIAA/USU Conference on Small Satellites*, Logan, USA, 2004.
- [10] S. Spangelo, A. Klesh, J. Cutler, Position and time system for the RAX small satellite mission, in: *AIAA/AAS Astrodynamics Specialist Conference*, 2010, p. 7980.
- [11] D. Oltrogge, A.G.I. Ramrath, A.G.I. Jens, Parametric characterization of SGP4 theory and TLE positional accuracy, in: S. Ryan (Ed.), *Proceedings of the Advanced Maui Optical and Space Surveillance Technologies Conference*, Wailea, Maui, Hawaii, September 9–12, The Maui Economic Development Board, 2014 id.E872014. <https://ui.adsabs.harvard.edu/abs/2014amos.confE..87O/abstract>.
- [12] J.R. Samson, Update on dependable multiprocessor CubeSat technology development, in: *2012 IEEE Aerospace Conference*, IEEE, Big Sky, MT, USA, 2012, pp. 1–12. <https://ieeexplore.ieee.org/abstract/document/6187238>.
- [13] D.L. Bekker, T.A. Werne, T.O. Wilson, P.J. Pingree, K. Dontchev, M. Heywood, R. Ramos, et al., A CubeSat design to validate the Virtex-5 FPGA for spaceborne image processing, in: *IEEE Aerospace Conference*, IEEE, Big Sky, MT, USA, 2010, pp. 1–9.
- [14] B. Denby, B. Lucia, Orbital edge computing: machine inference in space, *IEEE Comput. Archit. Lett.* 18 (1) (2019) 59–62.
- [15] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, A. Warfield, Xen and the art of virtualization, *ACM SIGOPS Oper. Syst. Rev.* 37 (5) (2003) 164–177.

- 
- [16] D. Lucchetti, S.K. Reinhardt, P.M. Chen, ExtraVirt: detecting and recovering from transient processor faults, in: *Proceedings of the Twentieth ACM Symposium on Operating Systems Principles*, 2005, pp. 1–8.
  - [17] S. Chien, R. Sherwood, D. Tran, B. Cichy, G. Rabideau, R. Castano, A. Davis, et al., Using autonomy flight software to improve science return on Earth Observing One, *J. Aerosp. Comput. Inf. Commun.* 2 (4) (2005) 196–216.